

PRÓLOGO

Quiero expresar mi agradecimiento a todas las personas que me han apoyado durante el desarrollo de este proyecto. Agradezco especialmente a mi profesor y tutor, Alfonso Rojas Bueno, por su guía y orientación a lo largo del curso, así como al resto del personal docente, que jugó igualmente un papel importante en la consecución de una formación integral, que contempla las necesidades y desafíos en el mercado laboral actual. También extendo mi gratitud a mi familia, a mis amigos, y a mis colegas informáticos por su constante apoyo, y por mantenerme siempre con ávida curiosidad sobre torno a tópicos relativos al desarrollo de software.

RESUMEN

Este proyecto consiste en el desarrollo de una aplicación web completa que integra tanto un frontend desarrollado con React como un backend construido con Django. El objetivo principal ha sido aplicar un stack tecnológico moderno que permita implementar funcionalidades reales de una plataforma tipo “films database”, incluyendo autenticación de usuarios, gestión dinámica de contenidos, comunicación en tiempo real mediante WebSockets, integración de pasarela de pago con Stripe, y una API RESTful robusta y segura.

La aplicación permite al usuario navegar por una base de datos de películas, consultar información detallada, interactuar con funcionalidades personalizadas y realizar operaciones autenticadas. Desde el punto de vista técnico, se ha trabajado con un enfoque de arquitectura desacoplada basada en cliente-servidor, donde el frontend consume exclusivamente servicios RESTful y canales WebSocket expuestos por el backend.

Se han empleado herramientas actuales del desarrollo profesional, tales como control de versiones con Git, despliegue en la nube mediante servicios como Vercel (frontend) y Render (backend), y virtualización de servicios mediante Docker. Además, se incorporaron patrones avanzados de desarrollo en React como el uso de hooks personalizados, contextos, stores y lógica compartida. Todo el proceso ha sido abordado de forma individual por un único desarrollador, con el objetivo de simular de forma realista el ciclo completo de

vida de un producto digital moderno: desde la planificación, análisis, diseño y desarrollo, hasta el despliegue y pruebas funcionales.

Keywords: React, Django, REST, WebSockets, Stripe, API, Frontend, Backend, Full Stack, Tailwind, Vite, JavaScript moderno, Django REST Framework, desarrollo web, arquitectura cliente-servidor, SPA (Single Page Application), despliegue en la nube, Docker, pruebas, integración continua, desarrollo ágil, SCRUM, control de versiones, GitHub, diseño responsivo.

ABSTRACT

This project involves the development of a complete web application that integrates a frontend built with React and a backend developed using Django. The main goal was to apply a modern technology stack capable of implementing real-world features commonly found in a “films database” platform, including user authentication, dynamic content management, real-time communication via WebSockets, payment gateway integration with Stripe, and a secure and robust RESTful API.

The application allows users to browse a movie database, view detailed information, interact with personalized features, and perform authenticated operations. From a technical perspective, the system follows a decoupled client-server architecture, where the frontend communicates solely through RESTful services and WebSocket channels provided by the backend.

Modern professional development tools were employed throughout the project, such as version control with Git, cloud deployment via Vercel (frontend) and Render (backend), and service virtualization using Docker. Advanced development patterns in React were also applied, including custom hooks, contexts, state stores, and shared logic. The entire process was handled individually by a single developer, aiming to simulate the full product development lifecycle: from planning, analysis, and design to implementation, deployment, and functional testing.

Keywords: React, Django, REST, WebSockets, Stripe, API, Frontend, Backend, Full Stack, Tailwind, Vite, Modern JavaScript, Django REST Framework, Web Development, Client-Server Architecture, SPA (Single Page Application), Cloud

Deployment, Docker, Testing, Continuous Integration, Agile Development, SCRUM, Version Control, GitHub, Responsive Design.

ÍNDICE

1. Portada.....	1
2. Prólogo	2
3. Resumen	2
4. Abstract	3
5. Índice	5
6. Introducción	7
7. Justificación / Motivación del TFG	7
8. Objetivos	8
8.1. Objetivos generales	
8.2. Objetivos específicos	
9. Estado del arte	9
10. Tecnologías utilizadas	10
11. Alternativas de tecnologías	12
12. Instalación de los softwares utilizados	13
13. Análisis del proyecto	15
13.1. Planeación / Planteamiento del problema	
13.2. Diagramas de entidad relación	
13.3. Arquitectura del proyecto	
14. Requisitos	28
14.1. Requisitos funcionales	
14.2. Requisitos no funcionales	
15. Despliegue	29
16. Presupuesto / Análisis de viabilidad	30
16.1. Costes de material	
16.2. Costes de personal	
17. Conclusiones personales	33

18. Pruebas	34
18.1. Pruebas de caja blanca	
18.2. Pruebas de caja negra	
19. Resultados obtenidos	34
20. Trabajos futuros / Líneas futuras	37
21. Bibliografía	38
22. Apéndices	39

INTRODUCCIÓN

En el presente documento se expone el desarrollo de un proyecto de fin de grado que tiene como finalidad la construcción de una aplicación web moderna, funcional y completa. Se ha abordado el desarrollo del frontend utilizando React, mientras que el backend se ha construido con Django, empleando una arquitectura basada en servicios REST. Además, se han integrado herramientas actuales del desarrollo profesional como WebSockets para comunicaciones en tiempo real y Stripe para la gestión de pagos.

Se trata de una plataforma destinada a los entusiastas del cine, que permite consultar datos detallados sobre películas, explorar información de fichas técnicas, géneros, reparto y otros metadatos relevantes. La aplicación también incluye funciones de diario personal, donde cada usuario puede llevar un registro de las películas que ha visto, sus valoraciones y reflexiones a modo de bitácora o log.

Además, incorpora elementos propios de una red social: los usuarios pueden interactuar entre sí, formar comunidades, compartir listas, comentar contenidos y seguir la actividad de otros cinéfilos. Esta fusión entre base de datos cinematográfica, herramienta de uso personal y entorno social convierte el proyecto en una propuesta integral, pensada para cubrir múltiples formas de interacción y uso en el ámbito del cine.

JUSTIFICACIÓN / MOTIVACIÓN DEL TFG

La motivación principal para llevar a cabo este proyecto ha sido el deseo de afrontar un reto integral dentro del ámbito del desarrollo web moderno, abarcando de forma equilibrada tanto el lado del cliente (frontend) como el del servidor (backend). Desde un enfoque autodidacta y práctico, el proyecto ha representado una oportunidad valiosa para adquirir experiencia en el diseño, construcción y despliegue de una arquitectura web completa, aplicando metodologías y herramientas que actualmente forman parte del entorno profesional.

De manera específica, el objetivo personal fue aprender, experimentar y consolidar conocimientos en tecnologías ampliamente adoptadas en la industria como React para la construcción de interfaces interactivas, Django REST

Framework para el diseño y consumo de APIs, WebSockets para habilitar comunicación bidireccional en tiempo real, y Stripe como solución para la integración de sistemas de pago. Cada una de estas herramientas fue seleccionada no solo por su popularidad, sino también por el valor que aportan en términos de escalabilidad, mantenibilidad y complejidad técnica.

Asimismo, el proceso ha servido como una simulación completa de un entorno de desarrollo real: desde la toma de decisiones arquitectónicas iniciales, la planificación y organización del trabajo, el desarrollo incremental de funcionalidades, la realización de pruebas manuales y técnicas, hasta la configuración de entornos de despliegue. Todo este recorrido ha sido ejecutado por un único desarrollador, lo que ha implicado la asunción de múltiples roles —analista, diseñador, programador, tester y administrador— reforzando así la capacidad de gestión autónoma de proyectos tecnológicos de cierta envergadura.

OBJETIVOS

Objetivos generales:

- Desarrollar una aplicación web completa y funcional utilizando tecnologías modernas.
- Aplicar buenas prácticas de arquitectura y organización de proyectos web.
- Acercamiento a herramientas diversas empleadas por distintos puestos o propósitos propios de un entorno laboral de IT.

Objetivos específicos:

- Implementar un frontend con React que consuma datos desde una API REST.
- Diseñar y desarrollar un backend con Django REST Framework.
- Integrar WebSockets para funcionalidades en tiempo real.
- Añadir sistema de pagos mediante Stripe.
- Gestionar el proyecto utilizando control de versiones y buenas prácticas de desarrollo.

- Documentar adecuadamente todo el proceso de desarrollo.

ESTADO DEL ARTE

Actualmente existen diversas plataformas centradas en la gestión, consulta y valoración de contenido cinematográfico. Una de las más representativas es Letterboxd, una aplicación social pensada para cinéfilos que permite registrar películas vistas, construir listas personalizadas, asignar valoraciones y redactar reseñas. Letterboxd ha logrado consolidar una comunidad activa gracias a un diseño visual atractivo, una experiencia de usuario fluida y una integración social que fomenta la interacción entre usuarios mediante seguidores, comentarios y actividad compartida.

Inspirado por el enfoque de Letterboxd, este proyecto persigue una visión similar, aunque más orientada al aprendizaje técnico y la extensibilidad futura. El sistema desarrollado intenta emular una versión simplificada y modular, concebida como una base funcional sólida desde la cual se puedan construir nuevas características escalables. A diferencia de soluciones comerciales como IMDb o FilmAffinity —más orientadas a la consulta de datos—, este proyecto ha sido íntegramente diseñado y codificado desde cero. Esto ha permitido un control absoluto sobre la arquitectura, las decisiones técnicas y la lógica de negocio, favoreciendo una comprensión profunda de cada uno de sus componentes.

Sin embargo, la propuesta va más allá de una mera réplica: contempla el aprovechamiento de oportunidades que plataformas actuales como Letterboxd no explotan del todo. En particular, se prevé potenciar la comunicación activa entre usuarios mediante funcionalidades que fomenten el diálogo, como mensajes privados, comentarios en tiempo real, foros temáticos y notificaciones. En un entorno donde miles de usuarios anónimos comparten una misma pasión, crear espacios de expresión accesibles y bien integrados es clave para consolidar una comunidad vibrante y participativa.

Además, el sistema ha sido diseñado con visión de producto. Esto incluye la posibilidad de extender la marca a través de líneas de merchandising oficial, aprovechando la integración de la pasarela de pagos Stripe, lo cual habilita de forma sencilla la monetización de productos o servicios relacionados. Este enfoque no solo refuerza la identidad del proyecto, sino que plantea oportunidades reales de escalado económico y comercial.

Por último, se ha considerado también la interoperabilidad. La aplicación ofrece la posibilidad de integrar y consumir servicios de terceros, como TMDb (The Movie Database) u OMDB (Open Movie Database), ya sea como fuentes de datos enriquecidos o incluso como parte de una futura API pública. Esta apertura no solo permite ampliar la información disponible para el usuario final, sino que posiciona a la plataforma como un posible nodo dentro del ecosistema digital de cine y medios.

TECNOLOGÍAS UTILIZADAS

Frontend:

- React 18
- Vite como bundler
- React Router para navegación
- Axios para peticiones HTTP
- Tailwind CSS para estilos
- Zustand para gestión de estado
- Whimsical para wireframing
- Node.js para gestión del entorno y dependencias
- Visual Studio Code como IDE para frontend

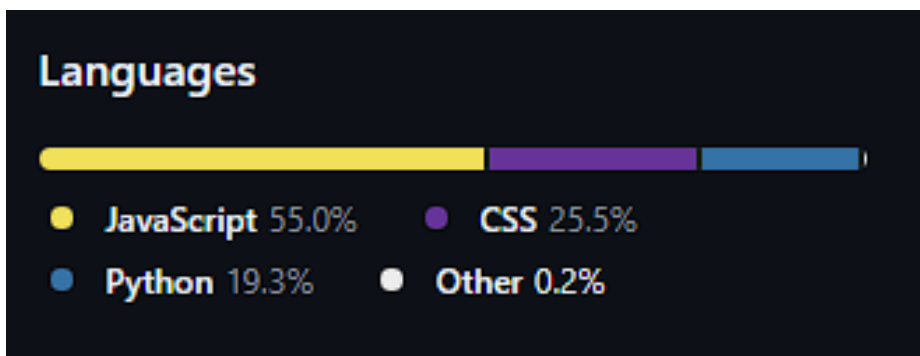
Backend:

- Python 3.12
- Django 5
- Django REST Framework
- SimpleJWT para autenticación
- Django Channels y Daphne para WebSockets
- Redis como broker de mensajes

- channels_redis
- PostgreSQL como base de datos
- Stripe como pasarela de pago
- Cloudinary para almacenamiento de archivos multimedia
- Elasticsearch y elasticsearch-dsl para búsqueda avanzada
- DRF Spectacular para documentación OpenAPI/Swagger
- django-cors-headers
- django-environ
- PyCharm como entorno de desarrollo para backend

Infraestructura y herramientas:

- Docker y Docker Compose para contenerización
- Git y GitHub para control de versiones
- Render para despliegue del backend
- Vercel para despliegue del frontend
- Postman para pruebas de la API
- Drawio para confección de diagramas
- Swagger para documentación API REST.



Métricas de código de la última versión en GitHub, previa a la presentación del presente documento.

ALTERNATIVAS DE TECNOLOGÍAS

A lo largo del desarrollo del proyecto se tomaron decisiones tecnológicas basadas en criterios de aprendizaje, eficiencia y compatibilidad. Cada herramienta y lenguaje fue seleccionado evaluando posibles alternativas en función de sus ventajas prácticas y su idoneidad para un entorno de desarrollo individual.

En el lado del frontend, se optó por utilizar **React** con un enfoque didáctico, al tratarse de una de las bibliotecas de JavaScript más demandadas en el mercado profesional. Aunque existen alternativas como Angular o Vue.js, React destaca por su flexibilidad, su ecosistema consolidado y su capacidad para optimizar el rendimiento mediante técnicas como el Virtual DOM, la carga perezosa de componentes y la reutilización modular del código. Además, su curva de aprendizaje —aunque exigente al inicio— proporciona una comprensión profunda del ciclo de vida de los componentes, lo que lo hace ideal para proyectos personales orientados al crecimiento técnico.

En cuanto al backend, se eligió el stack **Python–Django** frente a alternativas como Node.js (Express), Ruby on Rails o frameworks Java como Spring Boot. Django fue preferido por su enfoque “batteries-included”, es decir, por su amplio ecosistema de librerías integradas que resuelven necesidades comunes sin requerir una gran inversión en configuración inicial. Esto se traduce en una mayor agilidad durante el desarrollo y una disminución de la complejidad en proyectos individuales o de pequeño equipo. Python, además, ofrece una sintaxis clara y una gran comunidad activa, lo que facilita la resolución de problemas y la implementación rápida de nuevas funcionalidades.

Para la capa de persistencia de datos, se utilizó **PostgreSQL**, descartando otras opciones como MySQL o SQL Server. La decisión se basó en la excelente compatibilidad que PostgreSQL ofrece con Django, así como en su soporte nativo para tipos de datos complejos, consultas avanzadas y extensiones geoespaciales, lo que lo convierte en una base sólida para futuras ampliaciones del sistema.

En cuanto al diseño visual, se optó por **Tailwind CSS** en lugar de frameworks tradicionales como Bootstrap o Foundation. Tailwind se fundamenta en un enfoque utility-first que permite construir interfaces altamente personalizables sin necesidad de sobrescribir estilos predefinidos. Esta metodología proporciona un

mayor control sobre la maquetación, reduce el uso de archivos CSS personalizados y mejora la mantenibilidad del código, lo que resulta especialmente útil en desarrollos individuales donde se valora la eficiencia y la claridad.

Estas elecciones reflejan un criterio equilibrado entre exploración didáctica, rendimiento técnico y facilidad de integración, priorizando tecnologías modernas con fuerte soporte comunitario y potencial de escalabilidad.

Instalación de los softwares utilizados:

Para el desarrollo del proyecto se configuraron tanto el entorno backend como el frontend, así como servicios de apoyo mediante Docker. A continuación, se describen los pasos más relevantes y las tecnologías fundamentales que se instalaron durante el proceso de desarrollo.

En el backend, se creó un entorno virtual de Python utilizando el comando:

```
'python -m venv env'.
```

Una vez activado, se instalaron todas las dependencias definidas en el archivo requirements.txt mediante el comando

```
'pip install -r requirements.txt'.
```

Posteriormente, se inicializó el proyecto Django usando

```
'django-admin startproject',
```

y se crearon las aplicaciones internas con

```
'python manage.py startapp'.
```

Se ejecutaron las migraciones necesarias con 'makemigrations' y 'migrate', y el servidor de desarrollo se lanzó con:

```
'python manage.py runserver'.
```

Dentro de los paquetes más relevantes instalados para el backend se encuentran Django, Django REST Framework, django-cors-headers, drf-spectacular, django-enviro, así como channels y channels_redis para WebSockets. También se utilizaron librerías específicas para servicios integrados como stripe para la pasarela de pago, cloudinary para

almacenamiento multimedia, y elasticsearch con elasticsearch-dsl para funcionalidades de búsqueda avanzada.

En el frontend se utilizó React con Vite como entorno de desarrollo. El proyecto se inicializó con Vite, mediante el comando:

```
npm create vite@latest
```

Las dependencias se instalaron usando:

```
'npm install'.
```

La ejecución del servidor de desarrollo se realiza mediante:

```
'npm run dev'.
```

Se integraron bibliotecas esenciales como axios para las peticiones HTTP y zustand para la gestión de estado global.

Se utilizó Tailwind CSS como sistema de maquetación visual. Para su instalación no fue necesario configurar manualmente postcss ni tailwind.config.js, ya que Vite permite integrarlo directamente desde vite.config.js incluyendo la línea

```
'plugins: [react(), tailwindcss()]'.
```

Además, se estableció una configuración de alias para habilitar el uso de '@' como acceso rápido al directorio 'src', simplificando las importaciones. Finalmente, Tailwind se importó en el archivo global index.css, lo que permitió tener acceso a todas sus utilidades desde cualquier componente de la aplicación.

Para lanzar servicios auxiliares como PostgreSQL, Redis o Elasticsearch, se utilizó Docker con un archivo docker-compose. Desde consola, se ejecuta:

```
'docker-compose up'
```

para levantar todos los servicios necesarios en segundo plano. En sistemas Windows y macOS se requirió tener instalado Docker Desktop, y en el caso de Windows fue necesario contar con WSL2 habilitado.

Este proceso permitió tener un entorno de desarrollo consistente, reproducible y fácil de desplegar, tanto en local como en producción.

ANÁLISIS DEL PROYECTO

Planeación:

El desarrollo del proyecto siguió una metodología ágil de tipo SCRUM, con una estructura iterativa basada en sprints semanales y objetivos claramente definidos. Desde el inicio, se estableció una hoja de ruta orientada a entregar valor incremental a través de un producto mínimo viable (MVP), permitiendo validar el funcionamiento de los módulos esenciales antes de ampliar el alcance funcional.

El proceso comenzó con la identificación y documentación de los requisitos funcionales del sistema, lo cual permitió entender con claridad qué debía ofrecer la aplicación en su versión inicial. A partir de estos requisitos se diseñó el diagrama entidad-relación (ER), que estructuró la base de datos y sirvió como pilar para la lógica de negocio. Posteriormente, se elaboró un análisis técnico centrado en las tecnologías disponibles, el reparto de responsabilidades entre frontend y backend, y las posibles limitaciones en el entorno de desarrollo.

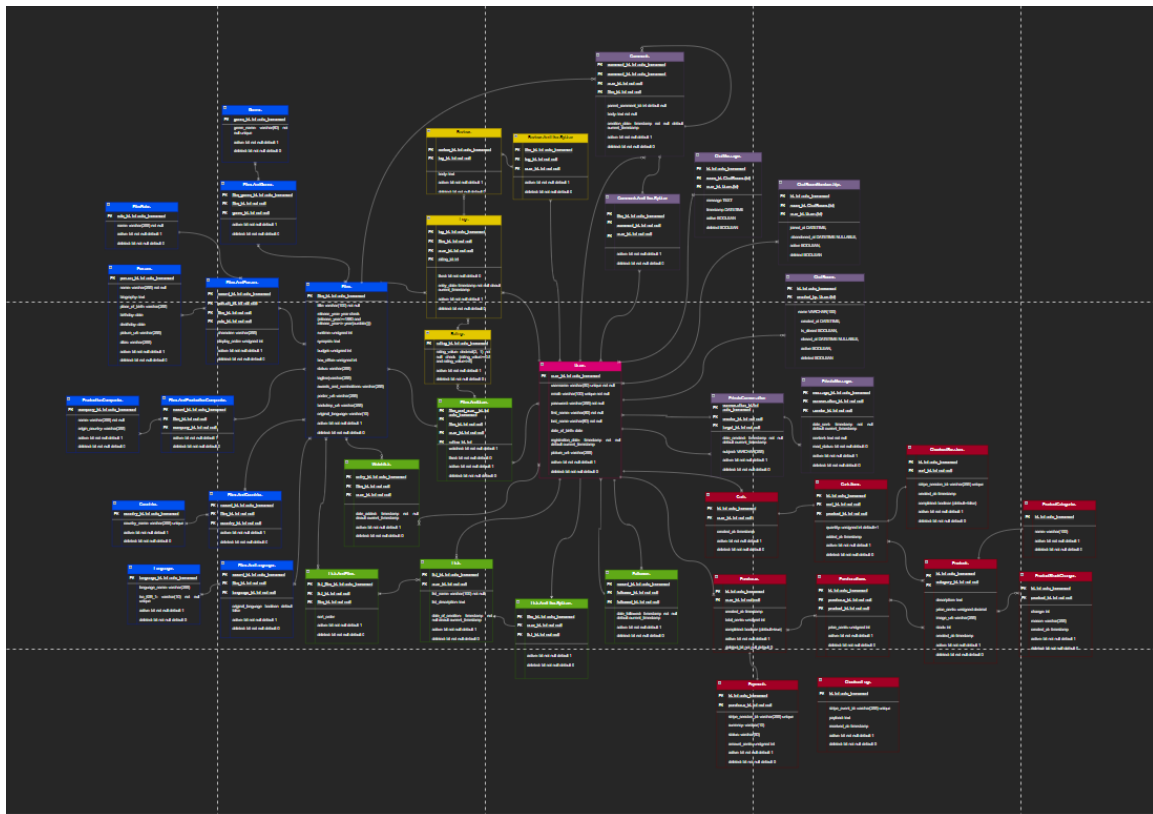
Una vez establecida la base técnica, se aplicó un enfoque de desarrollo nuclear, que consistió en construir primero los casos de uso más críticos y recurrentes para los usuarios: consulta de películas, navegación por fichas, visualización de detalles y gestión básica de listas. Este núcleo funcional garantizó un sistema coherente y usable desde las primeras versiones del proyecto.

Los sprints se planificaron con un criterio de prioridad y valor, enfocándose en liberar funcionalidades completas y operativas tras cada iteración. Este enfoque permitió mantener una visión clara del progreso, facilitar la toma de decisiones técnicas sobre la marcha y asegurar que el desarrollo avanzara con base en entregables concretos y medibles.

Diagramas:

-Diagrama de entidad-relación:

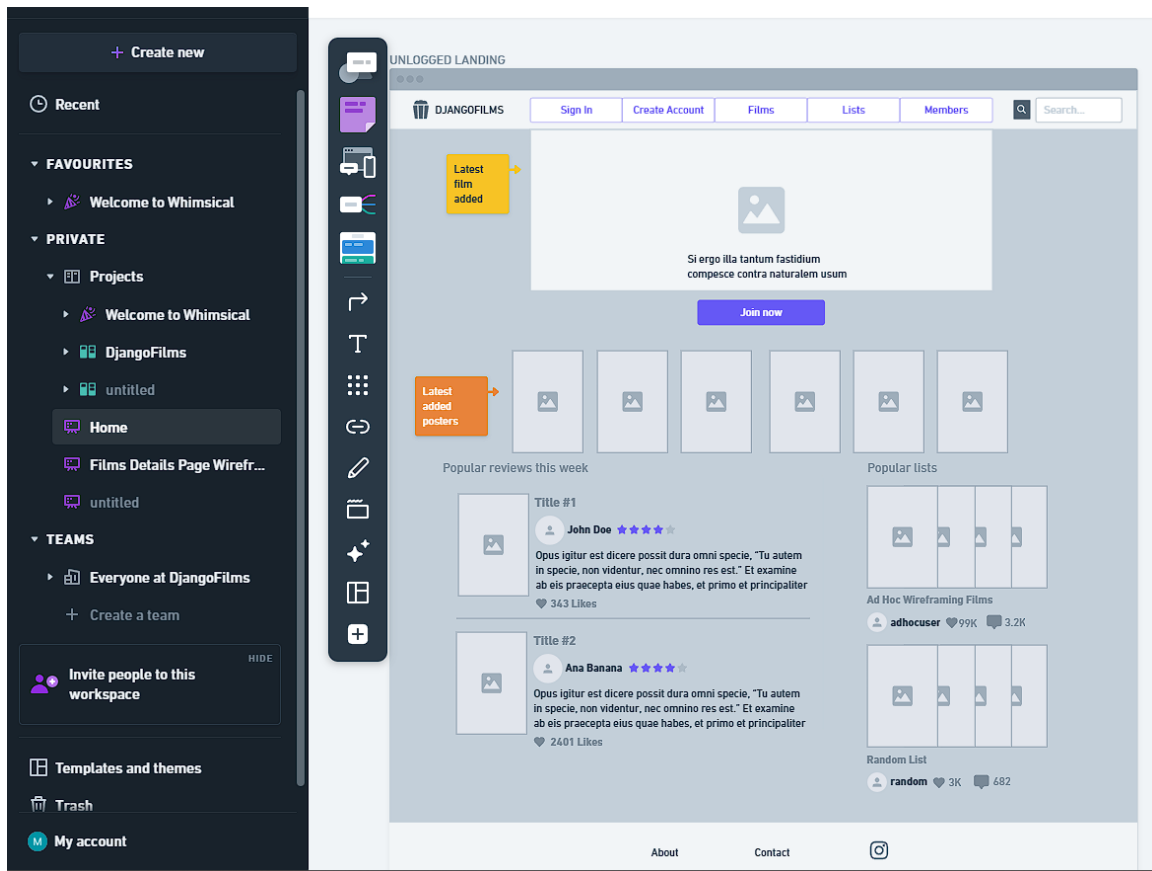
Se ha diseñado un diagrama ER que representa las relaciones entre usuarios, películas, géneros y listas. Este esquema facilitó el diseño de la base de datos y la estructura del backend.



Leyenda por módulo:

- Azul: Films.
- Verde: Users.
- Amarillo: Reviews.
- Rojo: Store.
- Púrpura: Comms.
- Rosa: Tabla maestra de usuarios proveída por Django Framework.

-Wireframes:



Whimsical como herramienta de mockups/wireframing.

Arquitectura del proyecto:

Scaffolding:

pDjangoFilmsV2/

- └─ .idea/ (configuración de proyecto de JetBrains/PyCharm)
- └─ activity/ (módulo transversal film-user-review sin modelos)
- └─ comms/ (comunicaciones: chat, sockets, mensajes)
- └─ core/ (clases base, middlewares, lógica común)
- └─ devtools/ (Docker, docker-compose, entorno local)
- └─ films/ (gestión de películas)
- └─ frontend/ (cliente React)
- └─ infra/ (documentación y archivos no técnicos)
- └─ integrations/ (Stripe, TMDB, OMDb)
- └─ pDjangoFilmsV2/ (settings, urls, wsgi de Django)
- └─ reviews/ (reseñas y puntuaciones)
- └─ search/ (búsqueda con Elasticsearch)
- └─ static/ (archivos estáticos manuales)
- └─ staticfiles/ (generado por collectstatic para producción)
- └─ store/ (carrito, órdenes, pagos)
- └─ users/ (registro, autenticación, perfiles)
- └─ utils/ (mocks y utilidades generales)
- |
- └─ .dockerignore (exclusiones para Docker)
- └─ .env (configuración local del entorno)
- └─ db.sqlite3 (base de datos de desarrollo)
- └─ manage.py (gestor de comandos Django)
- └─ requirements.txt (dependencias del backend)

frontend/

- └─ node_modules/ (dependencias instaladas por npm)
- └─ public/ (archivos públicos estáticos, como favicon o manifest)
- └─ src/ (código fuente de la aplicación React)
- |
- └─ .env.development (variables de entorno para desarrollo local)
- └─ .gitignore (archivos y carpetas ignoradas por Git)
- └─ eslint.config.js (configuración de reglas para linting del código)
- └─ index.html (archivo HTML principal de la aplicación)
- └─ jsconfig.json (configuración de rutas y alias en JavaScript)
- └─ package-lock.json (registro exacto de versiones de dependencias)
- └─ package.json (definición del proyecto, scripts y dependencias npm)
- └─ README.md (documentación base del proyecto)
- └─ src.zip (copia comprimida del código fuente del frontend)
- └─ vite.config.js (configuración de Vite y plugins como Tailwind)

src/

- └─ assets/ (archivos estáticos locales como imágenes, fondos y recursos gráficos)
- └─ components/ (elementos genéricos y reutilizables no ligados a dominios funcionales específicos)
- └─ features/ (conjuntos funcionales organizados por dominio de aplicación: listas, actividad, diarios, etc.)
- └─ hooks/ (hooks personalizados para lógica de red, sockets, integraciones o lógica compartida)
- └─ pages/ (vistas principales del sistema que estructuran secciones completas como usuario, inicio o búsqueda)
- └─ routes/ (configuración de navegación, rutas protegidas y condicionales según contexto)
- └─ services/ (clientes de red y lógica HTTP desacoplada: peticiones, control de errores, transformadores de respuesta)
- └─ store/ (estado global modularizado por área funcional, gestionado con Zustand)
- └─ utils/ (utilidades compartidas como validadores, formateadores, mocks, helpers funcionales)
- |
- └─ App.css (estilos generales de la aplicación)
- └─ App.jsx (componente raíz que orquesta la estructura y contexto de la app)
- └─ config.js (configuración general de constantes, rutas y entornos)
- └─ index.css (importación global de estilos, incluyendo Tailwind CSS)
- └─ main.jsx (punto de entrada que monta la app en el DOM)

Backend:

El sistema se construye sobre una arquitectura cliente-servidor desacoplada, donde el frontend desarrollado en React consume exclusivamente los recursos expuestos por el backend Django a través de una API RESTful bien definida. Adicionalmente, se integran WebSockets para permitir comunicación en tiempo real en ciertos casos de uso, como notificaciones u operaciones asincrónicas. Esta separación permite una escalabilidad limpia y favorece el desarrollo independiente de ambas capas.

El backend está estructurado según una arquitectura modular y basada en capas (N-layers). Cada capa tiene una responsabilidad claramente delimitada:

- Las vistas (ViewSets) actúan como capa de presentación y orquestación.
- La lógica de negocio se concentra en clases auxiliares, permisos, servicios, middlewares y validaciones.
- El acceso a datos queda delegado a los modelos Django, centralizados en parte bajo el módulo core.

Este diseño refleja una orientación práctica hacia el principio DRY (Don't Repeat Yourself), especialmente evidente en la herencia jerárquica de modelos. El módulo `core.models` define entidades abstractas reutilizables como `SoftDeleteModel` o `SoftManagedModel`, que implementan funcionalidades compartidas como el borrado lógico y la activación de registros. Estas clases son utilizadas de forma transversal por otros modelos del sistema, evitando duplicaciones y promoviendo la coherencia interna del ORM.

```

1  from django.db import models
2
3  class SoftDeleteModel(models.Model): 14 usages  ⤴ Matias
4      active = models.BooleanField(default=True)
5      deleted = models.BooleanField(default=False)
6
7      class Meta:  ⤴ Matias
8          abstract = True
9
10     class ActiveManager(models.Manager): 6 usages  ⤴ Matias
11         def get_queryset(self):  ⤴ Matias
12             return super().get_queryset().filter(active=True)
13
14     class SoftManagedModel(SoftDeleteModel): 28 usages  ⤴ Matias
15         objects = ActiveManager()
16         all_objects = models.Manager()
17
18         class Meta:  ⤴ Matias
19             abstract = True
20
21     class SoftCreateModel(SoftManagedModel): 2 usages  ⤴ Matias
22         class Meta:  ⤴ Matias
23             abstract = True
24
25

```

Definición en Core de clases abstractas adaptadas al borrado lógico.

```

19
20 class FilmAndUser(SoftCreateModel): 25 usages  ⤴ Matias
21     film_and_user_id = models.AutoField(primary_key=True, verbose_name="Film and User ID")
22     film = models.ForeignKey(Film, on_delete=models.CASCADE, verbose_name="Film")
23     user = models.ForeignKey(User, on_delete=models.CASCADE, verbose_name="User")
24     rating = models.ForeignKey(Rating, on_delete=models.SET_NULL, blank=True, null=True, verbose_name="Rating")
25     watched = models.BooleanField(default=True, verbose_name="Watched?")
26     liked = models.BooleanField(default=False, verbose_name="Liked?")
27

```

Implementación en un modelo crítico con requerimientos de borrado y creado lógico.

Además, se observa una preocupación constante por el aislamiento de responsabilidades, aplicando una separación explícita entre controladores (vistas), entidades del dominio (modelos), y políticas de seguridad (permisos, autenticación y middlewares). Este enfoque se alinea con los principios de clean

architecture, al permitir sustituir o extender componentes sin romper el funcionamiento global.

En lo que respecta al control de acceso, la aplicación implementa una arquitectura defensiva y multinivel, combinando tres capas principales:

-Middleware personalizado (OnlyReactAccessMiddleware), que actúa como firewall lógico. Inspecciona cabeceras específicas para permitir únicamente tráfico legítimo desde el frontend autorizado. Cuenta con mecanismos de lista blanca y negra, lo que permite abrir túneles específicos (como los requeridos por Stripe) sin comprometer la seguridad global.

```
3 class OnlyReactAccessMiddleware:
4     def __init__(self, request):
5         pass
6
7     def __call__(self, request):
8
9         # Allow OPTIONS
10        if method == "OPTIONS":
11            response = HttpResponse()
12        else:
13            # Protected internal routes (except webhook)
14            protected_paths = [
15                "/films/",
16                "/persons/",
17                "/users/",
18                "/reviews/",
19                "/activity/",
20                "/search/",
21                "/comms/",
22                "/store/",
23            ]
24
25            # Open pipeline for Stripe service
26            excluded_paths = ["/store/webhook/"]
27
28            if any(path.startswith(p) for p in protected_paths) and path not in excluded_paths:
29                if request.headers.get("X-Internal-Access") != "DjangoFilmsFrontend":
30                    return HttpResponseForbidden("Access denied")
31
32                response = self.get_response(request)
33
34            # Add CORS headers if coming from allowed frontend origins
35            if origin in [
36                "http://localhost:5173",
37                "http://localhost:5174",
38                "http://127.0.0.1:5173",
39            ]:
40                response["Access-Control-Allow-Origin"] = origin
41                response["Access-Control-Allow-Credentials"] = "true"
42                response["Access-Control-Allow-Headers"] = "Authorization, X-Internal-Access, Content-Type"
43                response["Access-Control-Allow-Methods"] = "GET, POST, PUT, DELETE, OPTIONS"
44
45            return response
```

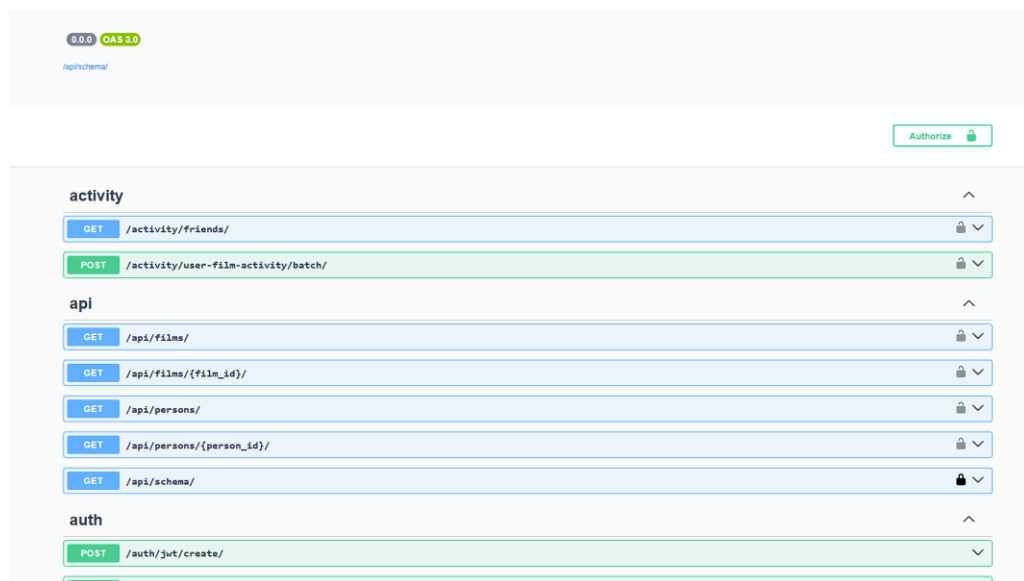
Túnel para subdominios mediante headers en protocolos HTTP.

-Gestión de CORS (mediante corsheaders), que restringe los orígenes permitidos a nivel de navegador.

-Autenticación por API Key (DRF + HasAPIKey), utilizada especialmente en rutas como /api/ destinadas a integraciones con terceros. Esta autenticación se complementa con declaraciones de permisos a nivel de vista, proporcionando un control granular del acceso según roles o funcionalidades.

El sistema también presenta un diseño con visión de extensibilidad: servicios como Clouinary, Stripe, Redis y Elasticsearch han sido integrados como capas externas desacopladas, preparadas para escalar o reemplazarse según las necesidades del entorno. La documentación automática de la API mediante DRF Spectacular y la gestión de configuración mediante django-environ refuerzan la mantenibilidad general del código.

En resumen, la arquitectura aplicada en el backend no solo es RESTful, sino también modular, segura, reutilizable y preparada para crecer. El uso de buenas prácticas como DRY, la separación en capas, la configuración desacoplada, y la construcción de componentes genéricos refleja un enfoque profesional en la planificación y ejecución técnica del sistema.



Documentación Swagger accesible en <http://localhost:8000/api/docs/>

Frontend:

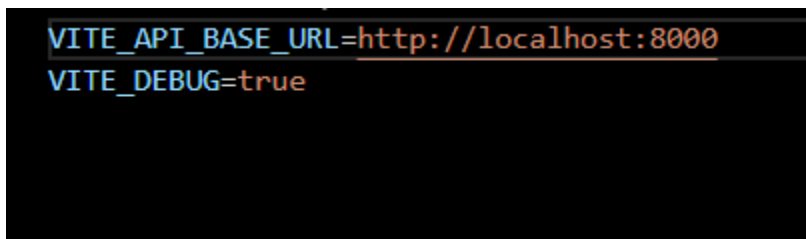
El frontend del sistema ha sido desarrollado utilizando React, estructurado sobre una arquitectura orientada a componentes y a funciones puras, en la que se fomenta la separación de responsabilidades, la reutilización de lógica, y la minimización del acoplamiento entre módulos. Esta capa consume de forma exclusiva la API RESTful proporcionada por el backend, actuando como cliente desacoplado bajo el patrón clásico de arquitectura cliente-servidor.

El proyecto adopta un enfoque moderno centrado en el uso de Zustand para la gestión de estado global, evitando el uso de soluciones más pesadas como Redux. Zustand permite mantener un modelo de estado reactivo, segmentado en *slices* independientes por dominio funcional (como useCartStore, userStore, listStore), lo que permite una organización modular del estado, alineada con el principio de responsabilidad única (SRP). Los stores encapsulan tanto los datos como las acciones necesarias para manipularlos, siguiendo un modelo explícito y controlado. En algunos casos, como en el store del carrito, se emplea persistencia con localStorage, garantizando la conservación del estado entre sesiones.

Desde el punto de vista del flujo de datos, el sistema utiliza prop drilling de forma controlada en componentes como FilmCard, en los que la transmisión de props desde componentes padres hacia elementos de presentación se mantiene estructurada y predecible. No obstante, para evitar la proliferación de props innecesarios y mejorar la escalabilidad, se delegan estados y lógica a los stores, facilitando el desacoplamiento entre componentes.

En cuanto a la comunicación con el backend, se implementa una capa de servicios desacoplada sobre Axios, compuesta por tres elementos clave:

-axios.js configura el cliente HTTP centralizado con base URL, cabeceras por defecto e interceptores.

A screenshot of a terminal window with a black background. It shows two lines of text in a monospaced font. The first line is 'VITE_API_BASE_URL=http://localhost:8000' and the second line is 'VITE_DEBUG=true'. The text is colored in a way that suggests it might be from a code editor or a terminal with syntax highlighting, with 'VITE_API_BASE_URL' and 'VITE_DEBUG' in blue and the values in orange.

Variable de entorno.


```

1  const BASE_URL = import.meta.env.VITE_API_BASE_URL;
2  const DEBUG = import.meta.env.VITE_DEBUG === "true";
3
4  if (!BASE_URL) {
5    throw new Error("VITE_API_BASE_URL is not defined in your environment");
6  }
7
8  export const config = {
9    BASE_URL,
10   DEBUG,
11 };

```

config.js

```

35  // Custom headers
36  request.headers["X-Internal-Access"] = "DjangoFilmsFrontend";
37
38  // Auth header
39  const token = getStoredAccessToken();
40  if (token) {
41    request.headers.Authorization = `Bearer ${token}`;
42  }
43
44  return request;
45  });
46
47  // --- Handle responses and token refresh --- //
48  api.interceptors.response.use(
49    (response) => {
50      const method = response.config.method?.toLowerCase();
51      const paramsKey =
52        method === "get" ? JSON.stringify(response.config.params || {}) : "";
53      const urlKey = `${method}:${response.config.url}:${paramsKey}`;
54

```

axios.js, responsable los headers y empleando config.js para que el back acepte peticiones.

-requestHandler.js actúa como orquestador de las peticiones, unificando patrones de consumo, control de errores y validaciones.

-exceptionHelper.js transforma errores técnicos en mensajes interpretables, contribuyendo tanto a la trazabilidad como a la experiencia de usuario.

```

1  import api from "./axios";
2  import { handleApiError } from "./exceptionHelper";
3
4  Complexity is 7 It's time to do something...
5  export const fetchData = async (url, { params = {}, signal = null } = {}) => {
6      try {
7          const res = await api.get(url, { params, signal });
8          return res.data;
9      } catch (err) {
10         if (err?.code === "ERR_CANCELED") return null;
11         console.error("[fetchData] RAW error:", err);
12         throw handleApiError(err);
13     }
14 };
15
16 Complexity is 7 It's time to do something...
17 export const postData = async (url, payload = {}, { signal = null } = {}) => {
18     try {
19         const res = await api.post(url, payload, { signal });
20         return res.data;
21     } catch (err) {
22         if (err?.code === "ERR_CANCELED") return null;
23         console.error("[postData] RAW error:", err);
24         throw handleApiError(err);
25     }
26 };
27
28 Complexity is 7 It's time to do something...
29 export const patchData = async (url, payload = {}, { signal = null } = {}) => {
30     try {
31         const res = await api.patch(url, payload, { signal });
32         return res.data;
33     } catch (err) {
34         if (err?.code === "ERR_CANCELED") return null;
35         console.error("[patchData] RAW error:", err);
36         throw handleApiError(err);
37     }
38 };

```

requestHandler.js implementando los protocolos HTTP mediante Axios.

```

18 export const getListDetails = (username, id, options = {}) =>
19   fetchData(`/users/${username}/lists/${id}/`, options);
20
21
22 export const createList = (username, payload, options = {}) =>
23   postData(`/users/${username}/lists/`, payload, options);
24
25 export const updateList = (id, payload, options = {}) =>
26   patchData(`/users/lists/${id}/`, payload, options);
27
28 export const deleteList = (id, options = {}) =>
29   deleteData(`/users/lists/${id}/`, options);

```

Ejemplo de llamadas API utilizando el requestHandler.js

Este enfoque sigue el patrón de Service Layer, desacoplando completamente la lógica de red del resto de la aplicación, y permitiendo aplicar buenas prácticas como DRY y clean code. Además, esta capa está pensada para extenderse con facilidad (por ejemplo, para logging, retries o métricas).

El frontend también hace uso de hooks personalizados que encapsulan lógica reutilizable (por ejemplo, en conexiones WebSocket o validación de formularios). Estos hooks abstraen comportamiento complejo en unidades reutilizables, alineándose con los principios de *composición funcional* y evitando duplicación.

La maquetación visual se basa en Tailwind CSS, un framework utility-first que permite construir interfaces de forma rápida y precisa, sin generar dependencias con estilos en cascada o sistemas de diseño rígidos. Esto otorga una gran flexibilidad al desarrollador para adaptar el diseño sin sacrificar orden o legibilidad.

Por último, herramientas como Whimsical fueron utilizadas en etapas previas para realizar wireframes de la interfaz, garantizando un enfoque planificado y visual desde la etapa de prototipado.

En conjunto, el frontend demuestra una arquitectura robusta, moderna y profesional. Se destacan prácticas como:

- Gestión de estado modular y eficiente con Zustand.
- Aislamiento claro de lógica de red mediante una capa de servicios
- Uso de hooks personalizados para encapsular y compartir lógica.

-Maquetación limpia y escalable con Tailwind CSS.

-Adopción consistente de principios como DRY, SoC (Separation of Concerns) y composición.

Todo ello contribuye a una base de código clara, mantenible, preparada para escalar en funcionalidades o en equipo, y alineada con estándares actuales del desarrollo web frontend.

REQUISITOS

Requisitos funcionales:

- Registro e inicio de sesión de usuarios.
- Consulta de información de películas.
- Gestión de listas personalizadas.
- Interacción en tiempo real mediante WebSockets.
- Procesamiento de pagos mediante Stripe.

Requisitos no funcionales:

- Navegación responsive en dispositivos móviles.
- Tiempo de respuesta inferior a 500ms para operaciones comunes.
- Base de datos local persistente en desarrollo, como PostgreSQL.
- Memoria RAM recomendada: 2GB o superior.
- Espacio en disco estimado: 100MB para entorno de pruebas.

DESPLIEGUE

Para un potencial despliegue se han previsto dos servicios populares:

- Render para el backend en Django.
- Vercel para el frontend React.

Dominios en entorno de desarrollo, atacando a los puertos convencionales por defecto:

-Django: <http://localhost:8000/>

-React: <http://localhost:5173/>

Dominios en entorno de desarrollo, atacando a los puertos convencionales por defecto:

<https://djangofilms-onrender.com>

<https://djangofilms.vercel.app>

El proyecto utiliza Git para el control de versiones, incluyendo ramas diferenciadas para desarrollo y producción. Se ha estructurado el repositorio con convenciones estándares que facilitan el mantenimiento y la colaboración futura. Las fases de despliegue contemplan entorno de pruebas, entorno de staging y entorno final de producción.

Enlace al repositorio:

<https://github.com/LiterallyNotABot/pDjangoFilmsV2>

La metodología empleada en cuanto al control de versiones, dado el scope del proyecto y la cantidad de desarrolladores involucrados (uno), invitó a un acercamiento sencillo: trabajar con dos ramas. Éstas serían Master y Development. Cualquier pull request se realizaría sobre development. Cada vez

que se obtenía una versión estable y cumpliendo los requisitos de viabilidad, se realizaría un merge hacia la rama maestra, manteniendo baja y estable la cantidad de versiones de la última.

PRESUPUESTO / ANÁLISIS DE VIABILIDAD

En cuanto a costes materiales, el proyecto no ha requerido inversión directa. Se ha desarrollado íntegramente desde un entorno doméstico, utilizando un ordenador personal y una conexión a internet ya disponibles. Las herramientas empleadas han sido de código abierto, y los servicios de despliegue (Render y Vercel) se han utilizado en sus versiones gratuitas, por lo que el coste económico real ha sido nulo.

No obstante, si se considera una proyección profesional del sistema —por ejemplo, como prototipo funcional de una plataforma tipo Letterboxd— se puede realizar un análisis de viabilidad estimando costes reales de desarrollo.

Este tipo de aplicación, con arquitectura REST, WebSockets, pasarela de pago integrada, almacenamiento en la nube, sistema de listas, perfiles, diarios, comunicación y consumo de APIs externas, puede ser abordado por un equipo reducido de dos desarrolladores (uno especializado en frontend y otro en backend).

Tomando en cuenta que el desarrollo completo ha requerido más de 80 horas individuales, y que en un contexto real se distribuiría la carga de trabajo, una estimación razonable para una versión inicial (MVP) podría ascender a:

-160 horas totales (80 horas por cada perfil: frontend y backend)

-Tarifa media de un desarrollador fullstack: 25 €/hora (junior) o 40 €/hora (semi-senior)

Escenario conservador (freelancers junior):

160 horas × 25 €/hora = 4.000 €

Escenario realista (perfil semi-senior):

160 horas × 40 €/hora = 6.400 €

A esto podrían sumarse costes operativos si el producto fuera a entrar en producción:

- Planes premium de Render/Vercel (según consumo): 20–50 €/mes
- Cloudinary profesional (multimedia): desde 89 €/mes
- ElasticSearch en servicio gestionado: desde 50 €/mes
- Stripe aplica comisiones por transacción (no implica gasto anticipado)

Estos costes no son necesarios en etapas de desarrollo, pero sí en una puesta en producción a escala real.

Además, se debe considerar el valor añadido del conocimiento adquirido durante el proceso. El dominio de tecnologías como React, Zustand, Docker, Django REST Framework, WebSockets y Stripe representa una base sólida para cualquier desarrollador que pretenda trabajar en entornos profesionales.

En conclusión, el proyecto no solo ha sido viable en su ejecución académica sin costes, sino que, en caso de evolución hacia una plataforma real en producción, se mantendría dentro de los parámetros económicos asumibles para una startup o equipo freelance pequeño con visión de producto.

En cuanto a la viabilidad de despliegue, si bien se han utilizado Render y Vercel en sus versiones gratuitas durante el desarrollo, en un escenario productivo pueden surgir necesidades de escalabilidad que impliquen costes mensuales. Para mantener el control presupuestario, se contemplan las siguientes alternativas de hosting más económicas:

-Railway:

Plataforma moderna que permite desplegar tanto frontend como backend con base en contenedores.

Ventajas: interfaz amigable, integración con bases de datos, buen rendimiento para Django y React.

Coste: plan gratuito con límites generosos, planes de pago desde 5–10 €/mes.

-Fly.io:

Permite ejecutar aplicaciones Dockerizadas en regiones globales.

Ventajas: rendimiento sólido, escalado automático, soporte para PostgreSQL y Redis.

Coste: gratuito hasta cierto uso mensual, luego basado en consumo (desde ~3 €/mes).

-Render:

Ya utilizado en el proyecto. El plan gratuito permite ejecución de servicios backend con durmientes automáticos.

Limitación: backend entra en modo sleep tras inactividad.

Planes pagos desde 7 €/mes.

-Vercel:

Utilizado para el frontend. El plan gratuito es más que suficiente para una SPA como la construida.

Ventaja: integración perfecta con React y Vite, despliegue con push a Git.

Coste: gratuito para desarrollo, planes profesionales desde 20 €/mes si se necesitan funciones avanzadas.

-VPS económicos (ej. Hetzner, Contabo, OVH):

En vez de usar plataformas PaaS, se puede optar por servidores virtuales económicos autogestionados.

Ejemplo: VPS en Hetzner desde 3–5 €/mes con 2 GB RAM, ideal para alojar servicios Docker, backend Django y base de datos.

-Github Pages + Cloudflare Pages (frontend estático):

Para despliegues puramente estáticos (frontend compilado con Vite), se puede usar Github Pages o Cloudflare Pages de forma gratuita y sin límite de peticiones.

CONCLUSIONES PERSONALES

El desarrollo de un proyecto con un alcance amplio como este requiere un proceso profundo de planificación, análisis y definición de arquitectura. La experiencia ha sido enriquecedora y retadora, especialmente en la adopción de React, una tecnología completamente nueva para el autor, que exige una forma de programación que a menudo parece opuesta a principios tradicionales como SOLID.

Lo que más me ha costado ha sido implementar React debido a su modelo de componentes, el manejo de estado y el enfoque declarativo que requiere una abstracción distinta a la programación imperativa clásica.

Objetivos cumplidos:

- Manejo de arquitectura REST.
- Integración de API de uso externo.
- Implementación de firewall lógico para proteger el acceso a la app.
- Implementación de herramientas complejas en React (stores, hooks, custom components).

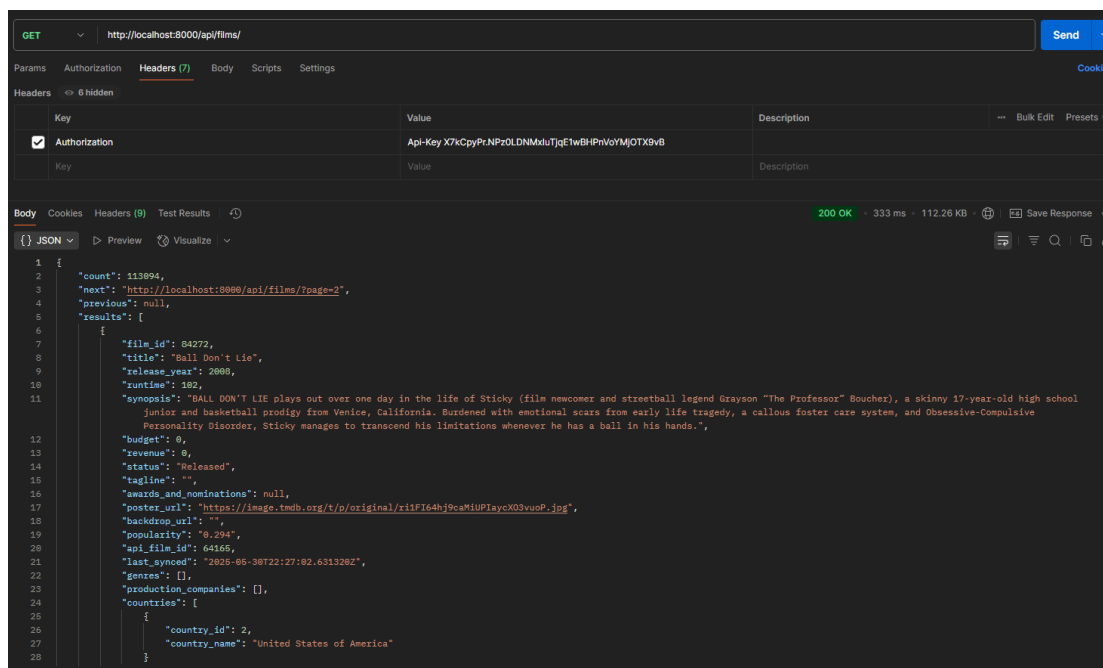
- Generación de herramientas de utilidad para el desarrollador (mockups, docker-compose, etc).

PRUEBAS

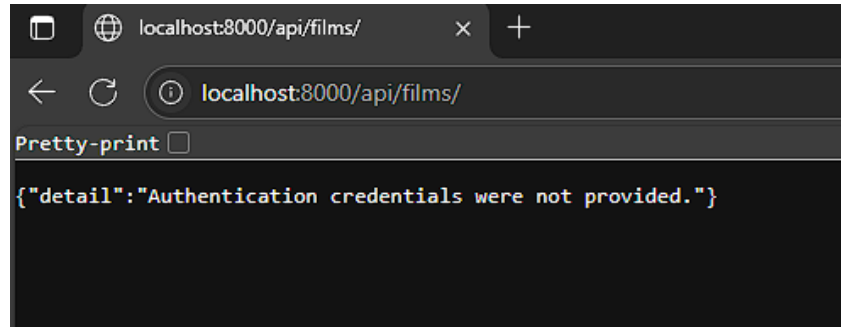
Se han realizado pruebas de caja blanca y caja negra de manera manual, centradas en los siguientes aspectos:

- Validación de entradas en formularios.
- Navegación entre rutas protegidas.
- Comprobación de flujos de usuario (registro, login, logout, uso de la app).
- Comprobación de llamadas API exitosas y fallidas.
- Verificación del comportamiento ante estados extremos (errores, sin datos, duplicados).

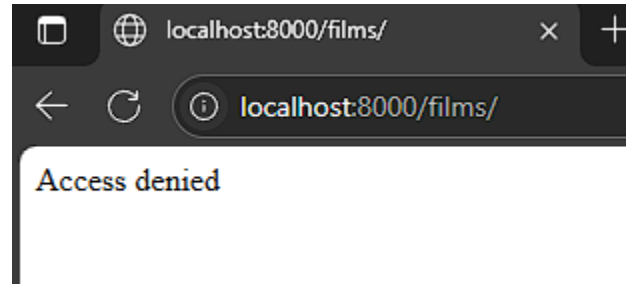
RESULTADOS OBTENIDOS



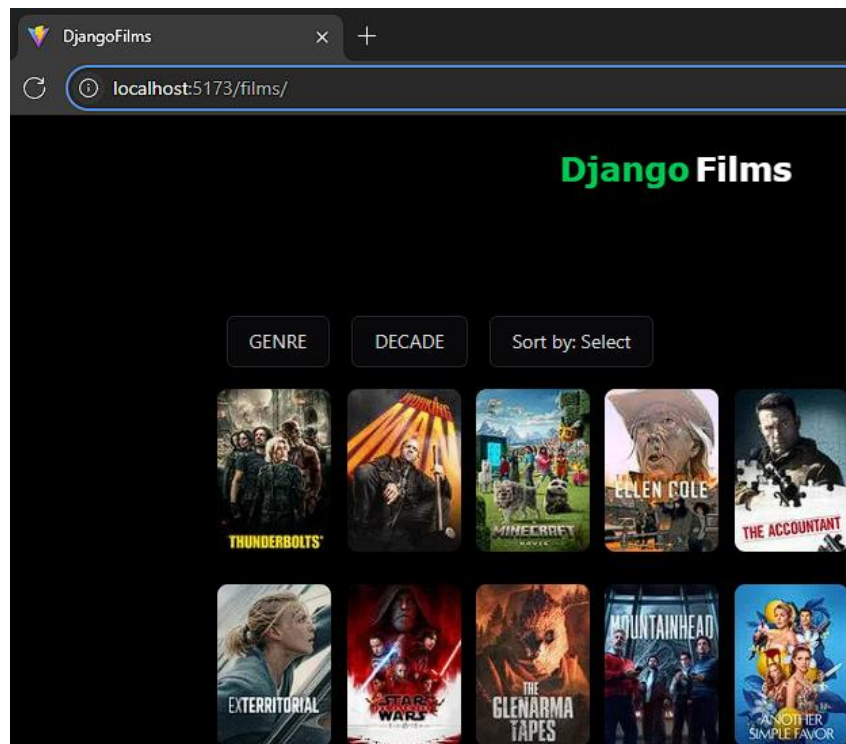
Petición a la API de uso "third party", utilizando API-Key en Postman.



Intento desde el navegador.



Intento de acceder a un subdominio no protegido por HasAPIKey de DRF, pero sí por el middleware diseñado.



Sólo accesible mediante en puerto 5173 en entorno local.

```

1  import time
2  import logging
3
4  logger = logging.getLogger(__name__)
5
6  class TimingMiddleware:
7      def __init__(self, get_response):
8          self.get_response = get_response
9
10     def __call__(self, request):
11         start_time = time.perf_counter()
12         response = self.get_response(request)
13         duration = time.perf_counter() - start_time
14
15         logger.info(f'{request.method} {request.get_full_path()} took {duration:.3f}s')
16
17         response["X-Request-Duration"] = f'{duration:.3f}s'
18         return response

```

Implementación de middleware para medir tiempos de respuesta de endpoints.

```

69
70 MIDDLEWARE = [
71     'django.middleware.security.SecurityMiddleware',
72     'django.contrib.sessions.middleware.SessionMiddleware',
73     'corsheaders.middleware.CorsMiddleware', # WARNING: CORS BEFORE COMMONMIDDLEWARE
74     'core.middlewares.react_access.OnlyReactAccessMiddleware',
75     'core.middlewares.timing_middleware.TimingMiddleware', # TESTING
76     'django.middleware.common.CommonMiddleware',
77     'django.middleware.csrf.CsrfViewMiddleware',
78     'django.contrib.auth.middleware.AuthenticationMiddleware',
79     'django.contrib.messages.middleware.MessageMiddleware',
80     'django.middleware.clickjacking.XFrameOptionsMiddleware',
81 ]
82

```

```

237
238 # TESTING
239 LOGGING = {
240     'version': 1,
241     'disable_existing_loggers': False,
242     'formatters': {
243         'verbose': {
244             'format': '{levelname} {asctime} {name} {message}',
245             'style': '{',
246         },
247     },
248     'handlers': {
249         'console': {
250             'class': 'logging.StreamHandler',
251             'formatter': 'verbose',
252         },
253     },
254     'loggers': {
255         'django': {
256             'handlers': ['console'],
257             'level': 'INFO',
258         },
259         'utils.middlewares.timing_middleware': {
260             'handlers': ['console'],
261             'level': 'INFO',
262             'propagate': False,
263         },
264     },
265     'root': {
266         'handlers': ['console'],
267         'level': 'WARNING',
268     },
269 }

```

Inclusión del middleware en settings.py

TRABAJOS FUTUROS / LÍNEAS FUTURAS

- Expansión del módulo de comunicaciones (mensajes privados, comentarios tipo foro, sistema de notificaciones y correos).
- Manejo de polling y restricciones en el uso de API externas.

- Ampliación del sistema de búsqueda.
- Mejora e integración del sistema de pagos en entorno de producción.
- Despliegue definitivo en entorno productivo.
- Desarrollo de herramientas adicionales para administración y monitorización.
- Mejoras en la maquetación y experiencia de usuario.
- Expansión del modelado de datos para incluir auditorías, históricos y eventos.
- Refuerzo en validaciones del backend, así como integración de clases helper reutilizables.

Otros posibles aportes:

- Internacionalización (i18n) de la aplicación.
- Integración de analíticas o dashboards.
- Automatización de despliegues con CI/CD.
- Inclusión de tests unitarios y de integración con herramientas como Jest o Pytest.

El proyecto queda como una base sólida para futuras iteraciones que podrían profesionalizarse a nivel de producto comercial.

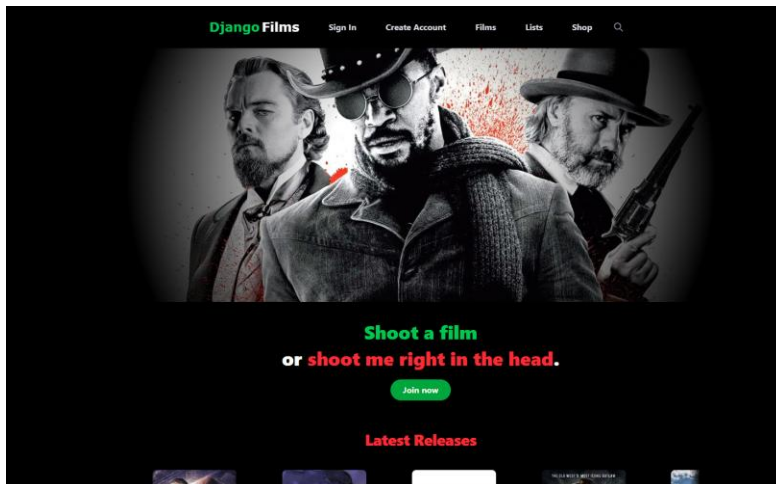
BIBLIOGRAFÍA

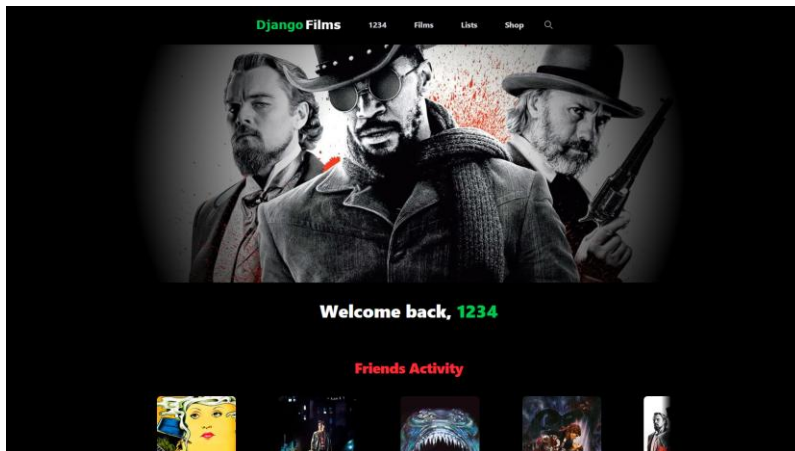
- React Documentation – <https://reactjs.org/docs/>
- Django Project Documentation – <https://docs.djangoproject.com/>
- Django REST Framework – <https://www.django-rest-framework.org/>
- Stripe API Reference – <https://stripe.com/docs/api>
- WebSockets MDN – https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

- Tailwind CSS Docs – <https://tailwindcss.com/docs>
- Git Documentation – <https://git-scm.com/doc>
- Vercel Documentation – <https://vercel.com/docs>
- Render Deployment – <https://render.com/docs>
- Stack Overflow – <https://stackoverflow.com/>

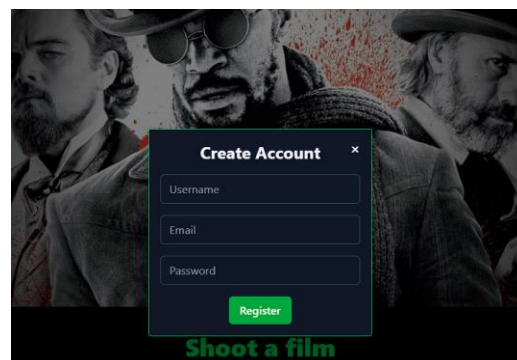
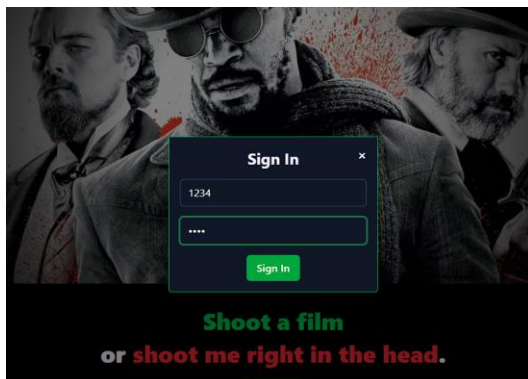
APÉNDICES

A continuación, se incluirán capturas de pantalla del proyecto en ejecución para ilustrar las distintas funcionalidades implementadas, así como ejemplos visuales de la interfaz de usuario, resultados de pruebas o despliegue.

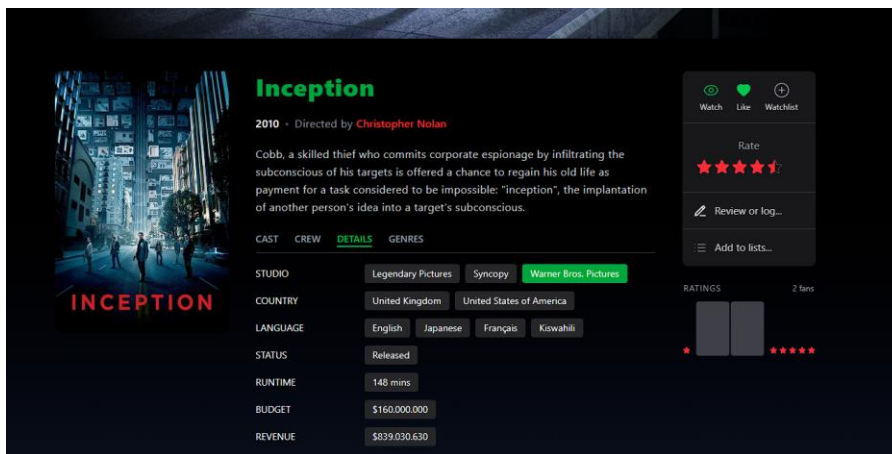
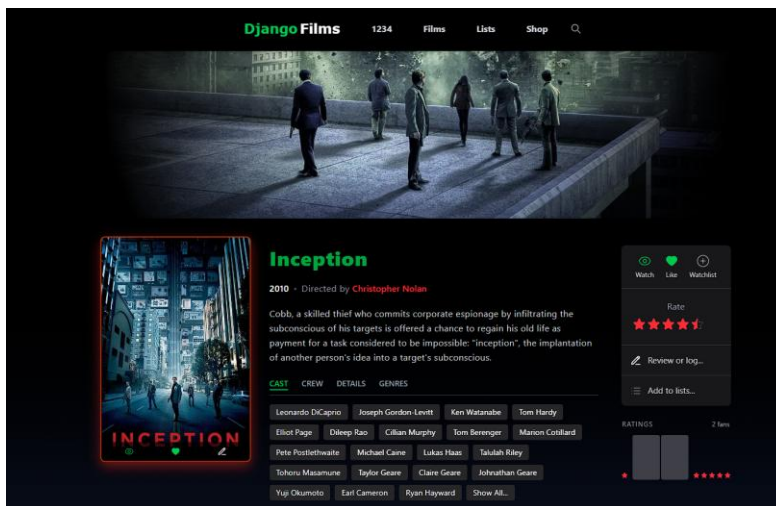




Home dinámico según haya user logeado o no (Latest Reviews vs. Friends Reviews).

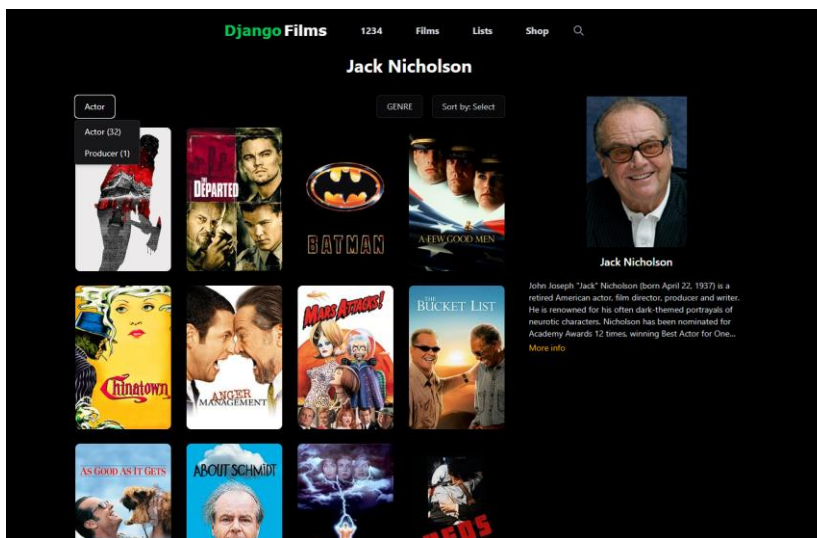
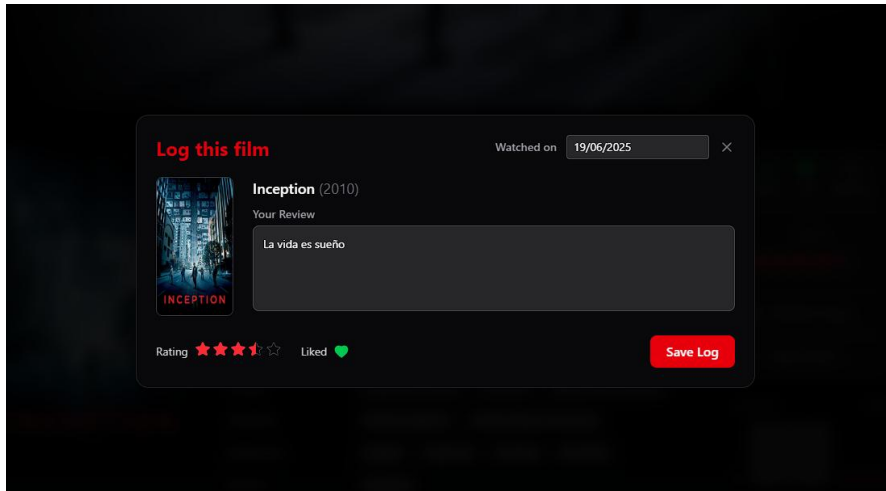


Modales de login/create user.

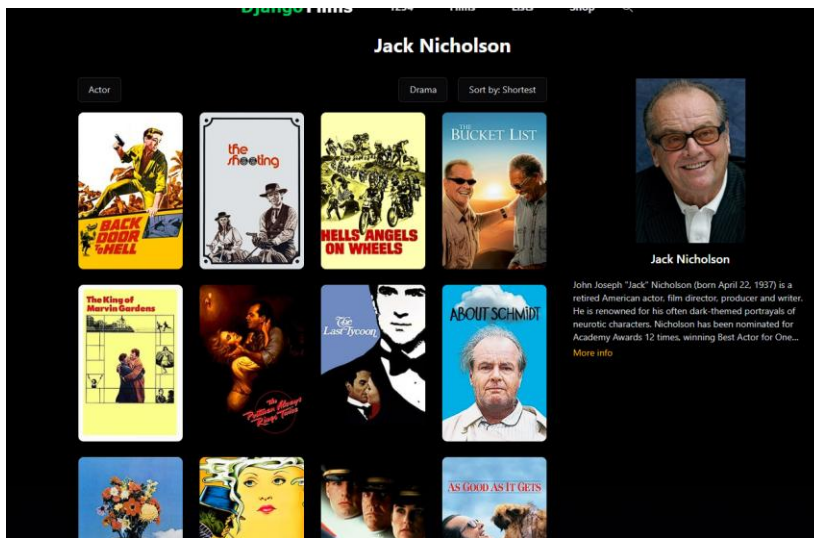


Se puede observar tarjeta con hover y dinámica. Si el user está logeado, se monta un componente de actividad dinámico al pie de la tarjeta. También, en la ficha técnica se la película, se observan distintos tabs organizados de forma lógica, así como un panel de acción de usuario, y métricas de ratings de la

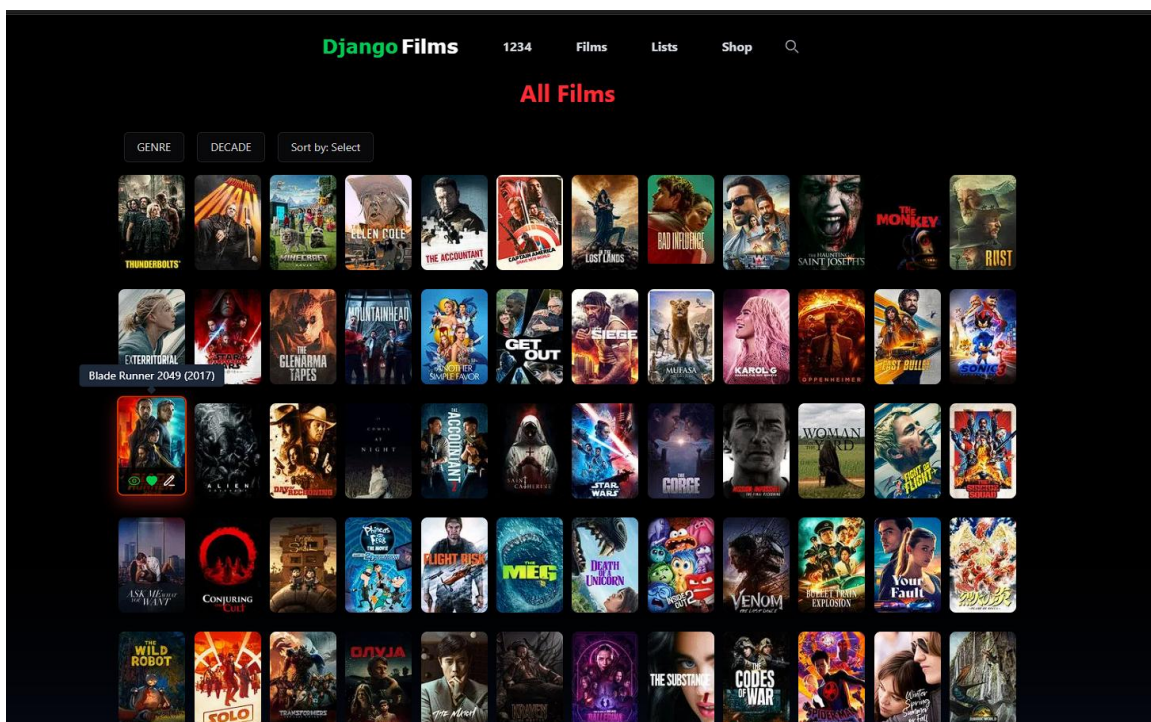
película. Si pulsamos log desde el panel de la derecha o bien desde el subcomponente de la tarjeta, tenemos un modal para dejar un log o reseña, con fecha elegible, rating, etc.

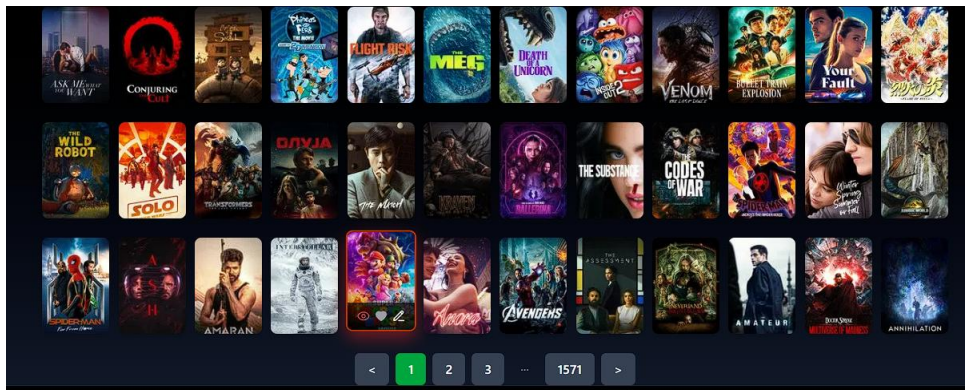


Acceso a perfil de persona, con sus películas asociadas, filtrables por rol de la persona y por género, y ordenables por diversos criterios.



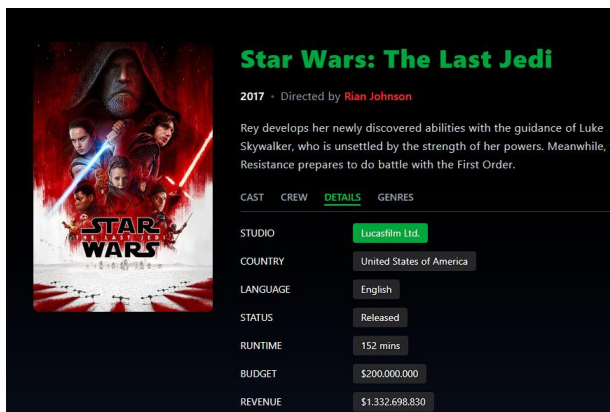
De modo que ahora tenemos las películas de drama de Jack Nicholson como actor, ordenadas por duración (primero las más cortas).



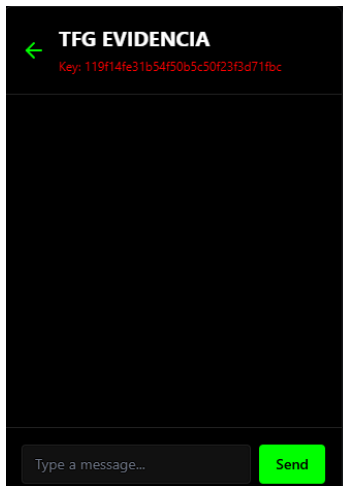
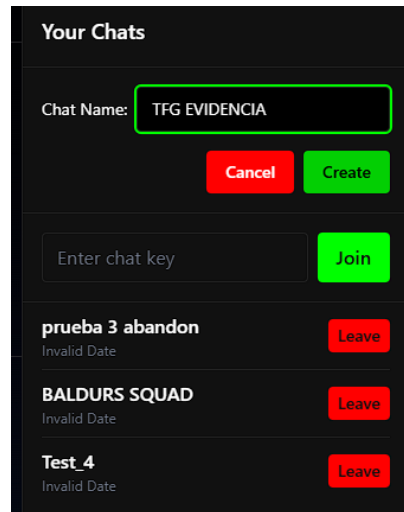
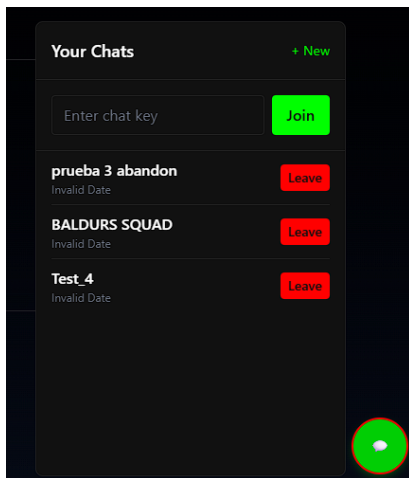


Tenemos grids grandes de películas con paginación y filtros estándar.

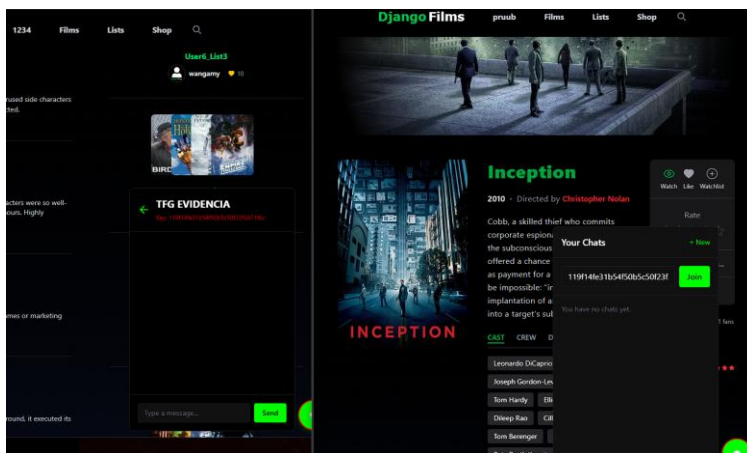
Sin embargo, podemos filtrar de formas más específicas seleccionando una entidad concreta. Ejemplo:

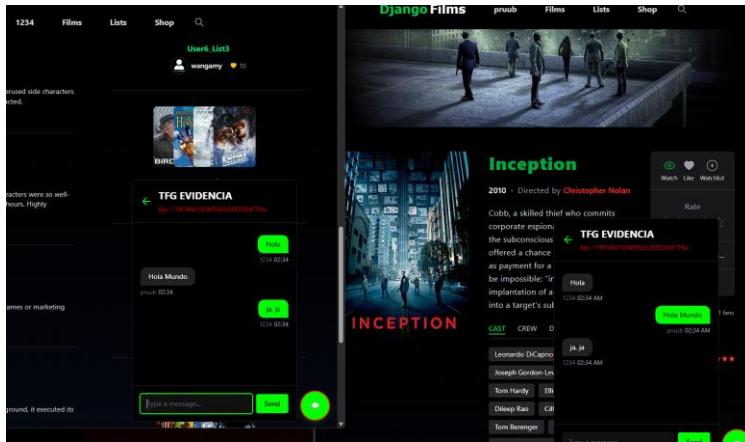


Al seleccionar el Badge de cierto estudio (en este caso LucasFilms), obtengo un grid con un filtro de entrada: sólo películas en cuyos estudios figure LucasFilms,

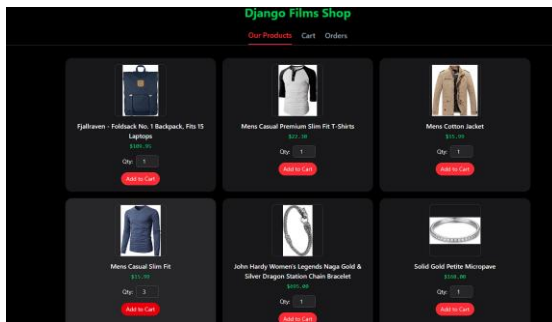


Como podemos ver, tenemos una lista de chatrooms a los que estamos suscritos. Al crear (o ingresar a) uno, se mostrará su respectiva clave. Ahora lo probaremos con 2 usuarios distintos (como refleja el navbar).





Tenemos también una tienda, poblada de productos mocks de una API gratuita llamada FakeStore.



Proceder con el carrito nos llevará a la pasarela de Stripe.

Entorno de prueba predeterminado

Entorno de prueba

Mens Casual Slim Fit

63,96 US\$

Cantidad 4, 15,99 US\$ unidad

link

4242

4242

Correo electrónico

matiaspadron@centroculturalalmantino.com

Información de la tarjeta

4242 4242 4242 4242

06 / 30

666

Nombre del titular de la tarjeta

MATIAS PADRON LOPEZ

País o región

España

Guardar mis datos de forma segura para un proceso de compra en un clic

Paga más rápido en Entorno de prueba predeterminado y en todos los comercios que acepten Link.

Pagar

Powered by stripe

Condiciones

Privacidad

Entorno de prueba predeterminado

Entorno de prueba

Mens Casual Slim Fit

63,96 US\$

Cantidad 4, 15,99 US\$ unidad

link

4242

4242

Correo electrónico

matiaspadron@centroculturalalmantino.com

Información de la tarjeta

4242 4242 4242 4242

06 / 30

666

Nombre del titular de la tarjeta

MATIAS PADRON LOPEZ

País o región

España

Guardar mis datos de forma segura para un proceso de compra en un clic

Paga más rápido en Entorno de prueba predeterminado y en todos los comercios que acepten Link.

Powered by stripe

Condiciones

Privacidad

Django Films

1234

Films

Lists

Shop

Django Films Shop

Our Products

Cart

Orders

Order #3

Mens Casual Slim Fit × 4

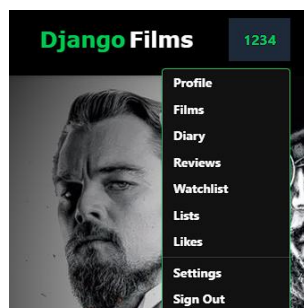
\$63.96

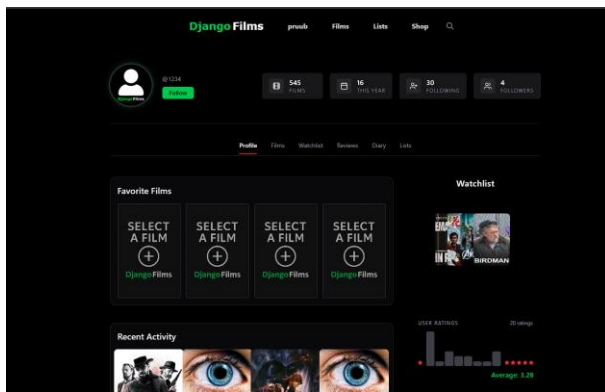
11/6/2025

Total: \$63.96

Pedido realizado con éxito.

Tenemos también nuestro dropdown que nos llevará a nuestro perfil.





@1234

[Settings](#)

FILMS

16 THIS YEAR

30 FOLLOWING

4 FOLLOWERS

[Profile](#)
[Films](#)
[Watchlist](#)
[Reviews](#)
[Diary](#)
[Lists](#)

Your Films

Sort by: Select

